The Journal of Open Source Software

Uno: a composable framework for nonlinearly constrained optimization

Alexis Montoison [1] **and Charlie Vanaret** [2]

1 Mathematics and Computer Science Division, Argonne National Laboratory, USA **2** Applied Optimization Department, Zuse-Institut Berlin, Germany

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Open Journals

Reviewers:

- @openjournals

Submitted: 01 January 1970
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

Uno is a composable software framework for nonlinearly constrained optimization written in modern C++. It unifies the workflows of Lagrange-Newton methods, i.e., gradient-based algorithms that iteratively solve the KKT optimality conditions with Newton's method. As of February 2026, Uno supports sequential (convex and nonconvex) quadratic programming, interior-point (barrier) methods, and sequential linear programming.

Uno breaks down optimization algorithms into reusable modular components such as step computation, constraint reformulation, globalization techniques, and acceptance criteria. This allows classical and hybrid methods to be configured and compared within a single framework.

The core C++ code of Uno is organized into modular, object-oriented components that separate the mathematical logic of the algorithms from implementation details such as memory management, data structures, and computational routines. For full mathematical details of the algorithms implemented in Uno, see (Vanaret & Leyffer, 2024). Uno also provides bindings for Julia, Python, C, and Fortran for use across scientific computing environments. Precompiled artifacts are available on GitHub, and the solver can be accessed directly via `UnoSolver.jl` in Julia or unopy in Python.

Statement of need

Nonlinearly constrained optimization is central to engineering, optimal control, machine learning, and scientific modeling (Nocedal & Wright, 2006). It also plays a key role in mixed-integer nonlinear optimization (MINLP) (Lee & Leyffer, 2011).

Popular solvers such as Ipopt (Wächter & Biegler, 2005), KNITRO (Byrd et al., 2006), and SNOPT (Gill et al., 2005) are efficient, but typically monolithic. They expose parameter tuning while keeping internal algorithmic components rigid and inaccessible. This limits algorithmic research, making it hard to prototype hybrid methods, systematically compare strategies, or teach the underlying techniques.

Uno addresses these gaps by providing a composable framework in which algorithms emerge from modular, code-level building blocks corresponding to mathematical concepts such as step computation, constraint reformulation, and globalization strategies. It enables rapid prototyping of new methods, serving both research and educational purposes.

Typical nonlinear solvers implement strategies such as sequential quadratic programming, interior-point methods, and augmented Lagrangian methods. In Uno, these strategies are organized into a wheel of layers and ingredients, illustrated in Figure 1. (TODO: use 2 different colors: one for implemented, one for not yet).

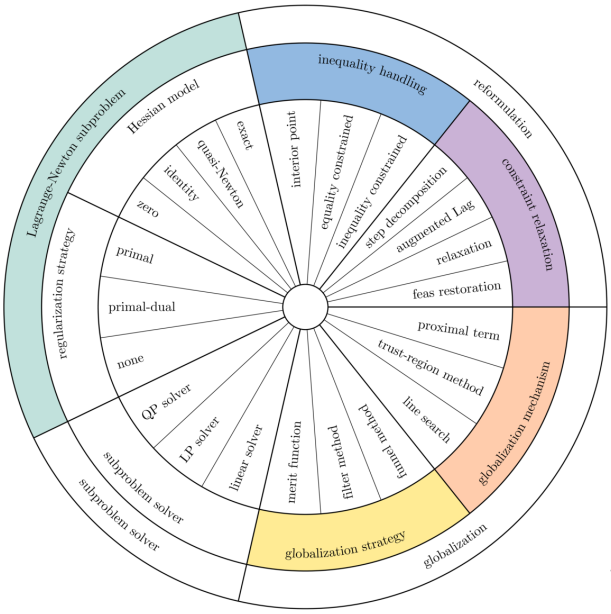

Figure 1: Unification framework: wheel of strategies.

Software design

The architecture of Uno follows a usual object-oriented design in which abstract classes define interfaces that should be implemented by subclasses. For instance, `BacktrackingLineSearch` and `TrustRegionMethod` both inherit from the abstract class `GlobalizationMechanism` and implement its interface.

Interfaces

To make Uno accessible to a wide range of users, we provide multiple language interfaces.

The first interface is based on AMPL (Fourer et al., 1990), a widely used modeling language for optimization problems. It is distributed as a binary that takes a compiled AMPL model (.nl file) as input, allowing users to solve problems without directly interacting with the C++ core.

The second interface is based on C and provides direct access to Uno's core functionality while maximizing interoperability with other programming languages and tools. It is centered around two main structures:

- **Model**: represents an optimization problem and stores information about variables, bounds, constraints, the objective function, and derivative information. Users create a model and set its components (objective, constraints, derivatives, initial iterates).
- **Solver**: represents the algorithm used to solve a given model. Users configure solver options, attach callbacks, and access results such as primal and dual solutions, residuals, iteration counts, and performance metrics.

The optimization phase requires both the model and the solver as input.

The third interface is based on Fortran and provides access to Uno through the C API using `iso_c_binding`. It is split into two files: `uno_c.f90` for low-level C bindings, and `uno_fortran.f90` for Fortran-friendly wrappers that handle string arguments. The interface can be included directly in a source file or wrapped in a module for cleaner `use` statements. It is

DRAFT

provided as source rather than a precompiled module to maximize portability and interoperability across compilers and platforms.

The fourth interface is based on Julia and is available as the registered Julia package `UnoSolver.jl`. It provides direct integration with the Julia optimization ecosystem through:

- a thin wrapper around the full C API,
- an interface to `NLPModels.jl` for solving problems following the NLPModels API, such as `ADNLPModels.jl` or `ExaModels.jl`,
- an interface to `MathOptInterface.jl` for handling JuMP models.

All precompiled artifacts are automatically downloaded, making the package plug-and-play without requiring compilation from users.

The fifth interface is based on Python and is available on PyPI as `unopy`. It provides access to Uno through precompiled wheels on most platforms, allowing users to define models, configure solvers, and retrieve solutions directly from Python.

A MATLAB interface is also under development, further expanding Uno's accessibility.

References

Byrd, R. H., Nocedal, J., & Waltz, R. A. (2006). KNITRO: An integrated package for nonlinear optimization. In *Large-scale nonlinear optimization* (pp. 35–59). Springer.

Fourer, R., Gay, D. M., & Kernighan, B. W. (1990). AMPL: A mathematical programming language. *Management Science*, *36*(5), 519–554.

Gill, P. E., Murray, W., & Saunders, M. A. (2005). SNOPT: An SQP algorithm for large-scale constrained optimization. *SIAM Review*, *47*(1), 99–131.

Lee, J., & Leyffer, S. (2011). *Mixed integer nonlinear programming*. Springer Science & Business Media.

Nocedal, J., & Wright, S. J. (2006). *Numerical optimization*. Springer.

Vanaret, C., & Leyffer, S. (2024). *Implementing a unified solver for nonlinearly constrained optimization*.

Wächter, A., & Biegler, L. T. (2005). Line search filter methods for nonlinear programming: Motivation and global convergence. *SIAM Journal on Optimization*, *16*(1), 1–31.

DRAFT